Lab Handout 2-E
Hardware Cyclic Redundancy Check (CRC)-8 and CRC-16 Verification

Capstone — Module 2: Field-Programmable Gate Array (FPGA) Register-Transfer Level (RTL) Design and Signal Ingestion

---

1 Objective

Add two hardware CRC engines and wire them into `pkt_parser`: a CRC-8 over the full interceptor forwarding frame (bytes 1–41) and a CRC-16/CCITT-FALSE over the embedded radio packet (frame bytes 10–39). Both engines run in a single clock cycle per byte. Frames whose CRCs do not match are diverted from `frame_valid` to `frame_reject`.

2 Prerequisites

- Lab Handout 2-D (`pkt_parser` accepting all length-correct frames as `frame_valid`).

3 Background

Two CRCs cover two different failure modes:

- **CRC-8** on the interceptor forwarding frame detects corruption between the STM32 interceptor and the FPGA Universal Asynchronous Receiver/Transmitter (UART). Polynomial $x^8 + x^2 + x + 1$ (`0x07`), initial value `0x00`, no reflection, no XOR-out (CRC-8/ITU).
- **CRC-16** embedded inside the radio packet detects over-the-air corruption end-to-end. Polynomial $x^{16} + x^{12} + x^5 + 1$ (`0x1021`, CRC-CCITT-FALSE), initial value `0xFFFF`, no reflection, no XOR-out.

Why not one CRC?

Two CRCs give independent visibility into where corruption happened. If CRC-16 fails but CRC-8 passes, the problem occurred over the air and the interceptor delivered the corrupted payload faithfully. If both fail, the serial link itself is suspect.

4 Procedure

4.1 Step 1 — Create `crc8_serial.vhd`

Sources panel: + → **Add or create design sources** → **Create File**, name `crc8_serial`. Replace its contents with the listing below and save.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity crc8_serial is
    port (
        clk      : in  std_logic;
        rst_n    : in  std_logic;
        crc_init : in  std_logic;                          -- pulse high to load 0x00
        byte_en  : in  std_logic;                          -- consume byte_in this cycle
        byte_in  : in  std_logic_vector(7 downto 0);
        crc_out  : out std_logic_vector(7 downto 0)
    );
end crc8_serial;

architecture rtl of crc8_serial is
    signal crc_r : std_logic_vector(7 downto 0) := (others => '0');

    function update8 (crc : std_logic_vector(7 downto 0);
                      d   : std_logic_vector(7 downto 0)) return std_logic_vector is
        variable c  : std_logic_vector(7 downto 0) := crc;
        variable fb : std_logic;
    begin
        for i in 7 downto 0 loop
            fb := c(7) xor d(i);
            c  := c(6 downto 0) & '0';
            if fb = '1' then
                c := c xor x"07";
            end if;
        end loop;
        return c;
    end function;
begin
    crc_out <= crc_r;
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                crc_r <= (others => '0');
            elsif crc_init = '1' then
                crc_r <= (others => '0');
            elsif byte_en = '1' then
                crc_r <= update8(crc_r, byte_in);
            end if;
        end if;
    end process;
end rtl;
```

Listing 1: `crc8_serial.vhd` — one-byte-per-cycle CRC-8/ITU (poly `0x07`, init `0x00`). Bit-serial update unrolled across 8 bits per byte, MSB-first.

4.2 Step 2 — Create `crc16_ccitt.vhd`

Same procedure: create `crc16_ccitt`, replace contents, save.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity crc16_ccitt is
    port (
        clk      : in  std_logic;
        rst_n    : in  std_logic;
        crc_init : in  std_logic;                          -- pulse high to load 0xFFFF
        byte_en  : in  std_logic;
        byte_in  : in  std_logic_vector(7 downto 0);
        crc_out  : out std_logic_vector(15 downto 0)
    );
end crc16_ccitt;

architecture rtl of crc16_ccitt is
    signal crc_r : std_logic_vector(15 downto 0) := (others => '1');

    function update16 (crc : std_logic_vector(15 downto 0);
                       d   : std_logic_vector(7 downto 0)) return std_logic_vector is
        variable c  : std_logic_vector(15 downto 0) := crc;
        variable fb : std_logic;
    begin
        for i in 7 downto 0 loop
            fb := c(15) xor d(i);
            c  := c(14 downto 0) & '0';
            if fb = '1' then
                c := c xor x"1021";
            end if;
        end loop;
        return c;
    end function;
begin
    crc_out <= crc_r;
    process (clk) begin
        if rising_edge(clk) then
            if rst_n = '0' then
                crc_r <= (others => '1');
            elsif crc_init = '1' then
                crc_r <= (others => '1');
            elsif byte_en = '1' then
                crc_r <= update16(crc_r, byte_in);
            end if;
        end if;
    end process;
end rtl;
```

Listing 2: `crc16_ccitt.vhd` — one-byte-per-cycle CRC-16/CCITT-FALSE (poly `0x1021`, init `0xFFFF`). Same shape as `crc8_serial` with a 16-bit state.

4.3 Step 3 — Hook the CRC engines into `pkt_parser`

Open `pkt_parser.vhd`. Replace its contents with the listing below (the LH2-D version augmented with CRC-8 over bytes 1–41 and CRC-16 over bytes 10–39, both checked in the `S_CAPTURE` branch for `byte_idx = 42`). Save.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pkt_parser is
    port (
        clk           : in  std_logic;
        rst_n         : in  std_logic;
        fifo_empty    : in  std_logic;
        fifo_data     : in  std_logic_vector(7 downto 0);
        fifo_rd_en    : out std_logic;
        feat_busy     : in  std_logic;
        metadata      : out std_logic_vector(63 downto 0);
        payload       : out std_logic_vector(255 downto 0);
        rx_crc        : out std_logic_vector(7 downto 0);
        frame_valid   : out std_logic;
        frame_reject  : out std_logic
    );
end pkt_parser;

architecture rtl of pkt_parser is
    type state_t is (S_HUNT, S_REQ, S_WAIT, S_CAPTURE, S_VALID, S_REJECT);
    signal state    : state_t := S_HUNT;
    signal byte_idx : integer range 0 to 63 := 0;

    signal meta_r    : std_logic_vector(63 downto 0)  := (others => '0');
    signal payload_r : std_logic_vector(255 downto 0) := (others => '0');
    signal crc_r     : std_logic_vector(7 downto 0)   := (others => '0');

    -- CRC-8 controls
    signal crc8_init : std_logic := '0';
    signal crc8_en   : std_logic := '0';
    signal crc8_byte : std_logic_vector(7 downto 0);
    signal crc8_val  : std_logic_vector(7 downto 0);

    -- CRC-16 controls
    signal crc16_init : std_logic := '0';
    signal crc16_en   : std_logic := '0';
    signal crc16_byte : std_logic_vector(7 downto 0);
    signal crc16_val  : std_logic_vector(15 downto 0);
begin
    metadata <= meta_r;
    payload  <= payload_r;
    rx_crc   <= crc_r;

    u_crc8 : entity work.crc8_serial
        port map (clk => clk, rst_n => rst_n,
                  crc_init => crc8_init, byte_en => crc8_en, byte_in =>
```

```
                    crc8_byte,
                crc_out => crc8_val);

    u_crc16 : entity work.crc16_ccitt
        port map (clk => clk, rst_n => rst_n,
                crc_init => crc16_init, byte_en => crc16_en, byte_in =>
                    crc16_byte,
                crc_out => crc16_val);

    process (clk) begin
        if rising_edge(clk) then
            frame_valid  <= '0';
            frame_reject <= '0';
            fifo_rd_en   <= '0';
            crc8_init    <= '0';
            crc8_en      <= '0';
            crc16_init   <= '0';
            crc16_en     <= '0';

            if rst_n = '0' then
                state    <= S_HUNT;
                byte_idx <= 0;
            else
                case state is
                    when S_HUNT =>
                        if fifo_empty = '0' and feat_busy = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                            byte_idx   <= 0;
                        end if;
                    when S_REQ =>
                        if fifo_empty = '0' then
                            fifo_rd_en <= '1';
                            state      <= S_WAIT;
                        end if;
                    when S_WAIT =>
                        state <= S_CAPTURE;
                    when S_CAPTURE =>
                        if byte_idx = 0 then
                            if fifo_data = x"7E" then
                                -- Initialize both CRCs at the START of
                                    byte 1.
                                crc8_init  <= '1';
                                crc16_init <= '1';
                                byte_idx <= 1;
                                state    <= S_REQ;
                            else
                                state <= S_HUNT;
                            end if;
                        elsif byte_idx = 1 then
                            -- Length byte. Feed CRC-8 (covers bytes
                                1..41).
                            crc8_byte <= fifo_data;
                            crc8_en   <= '1';
```

```
                    if fifo_data = x"29" then
                        byte_idx <= 2;
                        state    <= S_REQ;
                    else
                        state <= S_REJECT;
                    end if;
                elsif byte_idx >= 2 and byte_idx <= 9 then
                    meta_r(((byte_idx - 2) * 8 + 7) downto ((byte_idx - 2) * 8))
                        <= fifo_data;
                    crc8_byte <= fifo_data;
                    crc8_en   <= '1';
                    byte_idx  <= byte_idx + 1;
                    state     <= S_REQ;
                elsif byte_idx >= 10 and byte_idx <= 41 then
                    payload_r(((byte_idx - 10) * 8 + 7) downto ((byte_idx - 10) * 8))
                        <= fifo_data;
                    crc8_byte <= fifo_data;
                    crc8_en   <= '1';
                    -- CRC-16 covers radio bytes 0..29 = frame bytes 10..39.
                    if byte_idx <= 39 then
                        crc16_byte <= fifo_data;
                        crc16_en   <= '1';
                    end if;
                    byte_idx <= byte_idx + 1;
                    state    <= S_REQ;
                elsif byte_idx = 42 then
                    crc_r <= fifo_data;
                    -- CRC-8 check: captured trailing byte must equal running value.
                    --
                    -- CRC-16 reconstruction. The radio packet's embedded CRC-16
                    -- occupies radio bytes 30..31 = frame bytes 40..41. With byte 0
                    -- of the payload mapped to payload_r(7 downto 0), radio byte 30
                    -- lives in payload_r(247 downto 240) and radio byte 31 lives in
                    -- payload_r(255 downto 248). LH1-G transmits the high byte of
                    -- the CRC first (pkt_buf[30] = crc>>8, pkt_buf[31] = crc&0xFF),
                    -- so the original 16-bit value reconstructs as
                    --   (byte30 in MSByte) & (byte31 in LSByte)
                    -- = payload_r(247:240) & payload_r(255:248).
                    if (fifo_data = crc8_val) and
                       (payload_r(247 downto 240) & payload_r(255 downto 248) = crc16_val) then
                        state <= S_VALID;
                    else
```

```
                                state <= S_REJECT;
                            end if;
                        end if;
                    when S_VALID =>
                        frame_valid <= '1';
                        state       <= S_HUNT;
                        byte_idx    <= 0;
                    when S_REJECT =>
                        frame_reject <= '1';
                        state        <= S_HUNT;
                        byte_idx     <= 0;
                end case;
            end if;
        end if;
    end process;
end rtl;
```

Listing 3: `pkt_parser.vhd` (LH2-E revision) — length, CRC-8 frame, and CRC-16 radio-packet checks all gate `frame_valid`.

Where the captured CRC-16 lives in `payload_r`

The radio packet's embedded CRC-16 occupies radio bytes 30 and 31, which are frame offsets 40 and 41. With byte 0 of the payload mapped to `payload_r(7 downto 0)`, byte 30 is `payload_r(247 downto 240)` and byte 31 is `payload_r(255 downto 248)`. Module 1 transmits the high byte first (byte 30 = high byte, byte 31 = low byte), so the on-the-wire CRC-16 value reconstructs as `payload_r(247 downto 240) & payload_r(255 downto 248)`.

4.4 Step 4 — Golden-vector CRC testbench

Add a simulation source `tb_crc`. Replace its contents with the listing below.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_crc is
end tb_crc;

architecture sim of tb_crc is
    constant CLK_PERIOD : time := 10 ns;

    signal clk     : std_logic := '0';
    signal rst_n   : std_logic := '0';

    signal init8, en8   : std_logic := '0';
    signal byte8        : std_logic_vector(7 downto 0) := (others => '0');
    signal crc8         : std_logic_vector(7 downto 0);

    signal init16, en16 : std_logic := '0';
    signal byte16       : std_logic_vector(7 downto 0) := (others => '0');
    signal crc16        : std_logic_vector(15 downto 0);

```

```
    type byte_array_t is array (natural range <>) of std_logic_vector(7 downto 0);
    constant CHECK : byte_array_t := (
        x"31", x"32", x"33", x"34", x"35", x"36", x"37", x"38", x"39"
    );

    -- Local hex formatters. Vivado xsim does not always resolve to_hstring()
    -- for std_logic_vector under the default VHDL version, so build the hex
    -- strings by hand (same pattern used in LH2-B's tb_uart_rx).
    function slv_to_hex8 (v : std_logic_vector(7 downto 0)) return string is
        constant HEX : string(1 to 16) := "0123456789ABCDEF";
        variable hi  : integer := to_integer(unsigned(v(7 downto 4)));
        variable lo  : integer := to_integer(unsigned(v(3 downto 0)));
    begin
        return HEX(hi + 1) & HEX(lo + 1);
    end function;

    function slv_to_hex16 (v : std_logic_vector(15 downto 0)) return string is
    begin
        return slv_to_hex8(v(15 downto 8)) & slv_to_hex8(v(7 downto 0));
    end function;
begin
    clk <= not clk after CLK_PERIOD / 2;

    u_crc8  : entity work.crc8_serial
        port map (clk => clk, rst_n => rst_n,
                  crc_init => init8, byte_en => en8, byte_in => byte8, crc_out => crc8);

    u_crc16 : entity work.crc16_ccitt
        port map (clk => clk, rst_n => rst_n,
                  crc_init => init16, byte_en => en16, byte_in => byte16, crc_out => crc16);

    process
    begin
        wait for 100 ns;
        rst_n <= '1';
        wait until rising_edge(clk);

        init8  <= '1';
        init16 <= '1';
        wait until rising_edge(clk);
        init8  <= '0';
        init16 <= '0';

        for i in CHECK'range loop
            byte8  <= CHECK(i);
            byte16 <= CHECK(i);
            en8    <= '1';
```

```
            en16   <= '1';
            wait until rising_edge(clk);
        end loop;
        en8  <= '0';
        en16 <= '0';
        wait until rising_edge(clk);

        assert crc8 = x"F4"
            report "CRC-8 check failed: got 0x" & slv_to_hex8(crc8)
               severity failure;
        assert crc16 = x"29B1"
            report "CRC-16 check failed: got 0x" & slv_to_hex16(crc16)
               severity failure;

        report "tb_crc PASS" severity note;
        std.env.finish;
    end process;
end sim;
```

Listing 4: `tb_crc.vhd` — streams the ASCII string "123456789" through both CRC engines. The expected outputs are the canonical check values: CRC-8/ITU = `0xF4`, CRC-16/CCITT-FALSE = `0x29B1`.

Set `tb_crc` as the simulation top (right-click in Sources → Simulation Sources → **Set as Top**), launch behavioral simulation, then in the Tcl console type `run 5 us` (once) and press Enter. The test feeds 9 bytes through both CRC engines and checks the canonical "123456789" test vectors; total simulated time is well under 1 μs. Confirm `tb_crc PASS` prints in the Tcl console.

`to_hstring` portability note

Vivado xsim under the default VHDL-93 mode does not always resolve `to_hstring` for `std_logic_vector`. If compilation fails on the assertion-message lines (`type error near crc8 / crc16 ... expected type bit_vector`), replace each `to_hstring(...)` call with a small local helper that hand-builds the hex string from the vector's nibbles, mirroring the `slv_to_hex` pattern from LH2-B's `tb_uart_rx`.

4.5 Step 5 — Re-synthesize and confirm hardware

1. **Confirm `ingest_top` is set as the Design Sources top.** In the Sources panel's *Design Sources* tab, `ingest_top` must appear in **bold**. If a testbench or another entity is bold instead, right-click `ingest_top` → **Set as Top** before continuing.

2. Run synthesis. Expect zero errors.

3. Re-program the board. The seven-segment Beacon ID display should still update, but only frames whose CRCs match will increment the counter. `LED[4]` (`frame_reject`) should still be dark on a clean stream.

4. If `LED[4]` blinks frequently, your interceptor firmware (LH1-G) may be emitting incorrect CRC bytes — re-flash the Threat and re-confirm the LH1-G USART2-console behavioral validation, then re-run `tb_pkt_parser_crc` against the regenerated capture.

[Figure placeholder]

`LH2E_crc_coverage.png`

Byte coverage for each CRC engine. CRC-8 covers bytes 1–41; CRC-16 covers bytes 10–39. Bytes 40–41 carry the embedded CRC-16; byte 42 carries the CRC-8.

Figure 1: Byte coverage for each CRC engine. CRC-8 covers bytes 1–41; CRC-16 covers bytes 10–39. Bytes 40–41 carry the embedded CRC-16; byte 42 carries the CRC-8.

5 Verification Checklist

- ☐ `crc8_serial.vhd`, `crc16_ccitt.vhd`, and updated `pkt_parser.vhd` compile with zero errors.
- ☐ `tb_crc` prints `PASS`: CRC-8 = `0xF4` and CRC-16 = `0x29B1` for the canonical check string "123456789".
- ☐ `LED[4]` stays dark on a clean live capture.
- ☐ Post-synthesis timing for `pkt_parser` has positive slack (the CRC XOR trees are short; if WNS is negative, see LH2-H Step 2).

Lab Handout 2-E Deliverable

1. `crc8_serial.vhd`, `crc16_ccitt.vhd`, and updated `pkt_parser.vhd`.
2. `tb_crc.vhd` and Tcl console transcript showing `PASS`.
3. Hardware screenshot showing the seven-segment display updating with `LED[4]` dark over a 30 s clean run.

Glossary

CRC	Cyclic Redundancy Check	1, 4, 7, 9, 10
FPGA	Field-Programmable Gate Array	1
RTL	Register-Transfer Level	1
UART	Universal Asynchronous Receiver/Transmitter	1